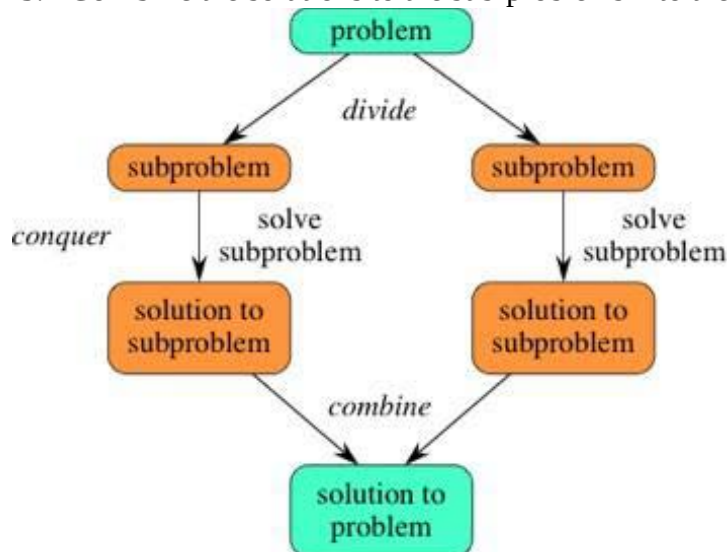


Module III

- **Divide & Conquer and Greedy Strategy**
 - **The Control Abstraction of Divide and Conquer**
 - **2-way Merge sort**
 - **Strassen's Algorithm for Matrix Multiplication-Analysis**
 - **The Control Abstraction of Greedy Strategy**
 - **Fractional Knapsack Problem**
 - **Minimum Cost Spanning Tree Computation- Kruskal's Algorithms – Analysis**
 - **Single Source Shortest Path Algorithm - Dijkstra's Algorithm-Analysis**
- **Divide and Conquer**
 - Divide and conquer algorithm is having three parts:
 1. **Divide** the problem into a number of sub-problems that are smaller instances of the same problem.
 2. **Conquer** the sub-problems by solving them recursively. If they are small enough, solve the sub-problems as base cases.
 3. **Combine** the solutions to the sub-problems into the solution for the original problem.



- Control Abstraction: It is a procedure whose flow of control is clear but whose primary operations are specified by other procedure whose precise meanings are left undefined.
- **Control Abstraction: Divide and Conquer**

```

Algorithm DAndC(P)
{
    if Small(P) then
        return S(P)
    else
    {
        Divide P into smaller instances  $P_1, P_2, \dots, P_k, k \geq 1$ ;
        apply DAndC to each of these sub-problems;
        return Combine(DAndC( $P_1$ ), DAndC( $P_2$ ),  $\dots$ , DAndC( $P_k$ ));
    }
}
  
```

- If the given problem is small, return the result
- Otherwise, divide the problem into smaller instances $P_1, P_2, \dots, P_k$
- Apply DAndC() to each of these sub-problems.
- Finally combine the results of all sub-problems.

- DAndC() can be described using the following recurrence relation:
- $$T(n) = \begin{cases} g(n) & n \text{ is small} \\ T(n_1) + T(n_2) + \dots + T(n_k) + f(n) & \text{Otherwise} \end{cases}$$
 - $T(n)$: Time for divide and conquer on any input of size n
 - $f(n)$: Complexity of dividing the problem and combining the results.

- Complexity of many divide and conquer algorithms are given by the following recurrence relation
- $$T(n) = \begin{cases} T(1) & n = 1 \\ aT(n/b) + f(n) & n > 1 \end{cases}$$

o 2 Way Merge Sort

- Given a sequence of n elements $a[1], \dots, a[n]$. Split this array into two sets $a[1], \dots, a[n/2]$ and $a[(n/2)+1], \dots, a[n]$. Each set is individually sorted, and the resulting sorted sequences are merged to produce a single sorted sequence of n element.

```

Algorithm MergeSort(low, high)
{
    mid = (low + high )/2;
    MergeSort(low, mid);
    MergeSort(mid+1, high);
    Merge(low, mid, high);
}

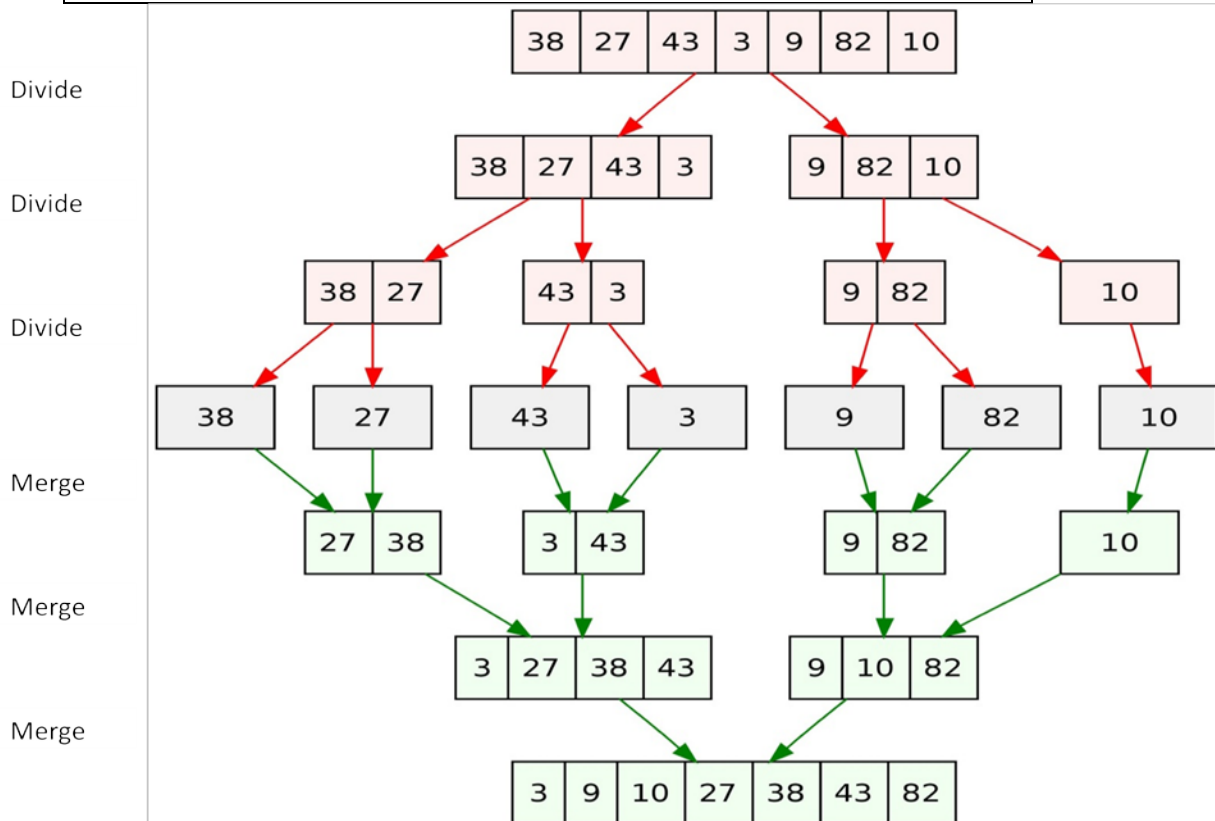
Algorithm Merge(low, mid, high)
{
    i= low; x= low; y= mid + 1;
    while((x ≤ mid) and (y ≤ high)) do
    {
        if ( a[x] ≤ a[y] ) then
        {
            b[i] = a[x];
            x = x+1;
        }
        else
        {
            b[i] = a[y];
            y = y+1;
        }
        i=i+1;
    }
}

```

```

if(  $x \leq mid$ ) then
{
    for  $k=x$  to  $mid$  do
    {
         $b[i] = a[k];$ 
         $i = i+1;$ 
    }
}
else
{
    for  $k=y$  to  $high$  do
    {
         $b[i] = a[k];$ 
         $i = i+1;$ 
    }
}
for  $k= low$  to  $high$  do
     $a[k] = b[k];$ 
}

```



- Complexity**

$$T(n) = \begin{cases} a & \text{if } n=1 \\ 2T(n/2) + cn & \text{Otherwise} \end{cases}$$

a is the time to sort an array of size 1

cn is the time to merge two sub-arrays

$2T(n/2)$ is the complexity of two recursion calls

$$\begin{aligned}
T(n) &= 2 T(n/2) + c n \\
&= 2(2 T(n/4) + c(n/2)) + c n \\
&= 2^2 T(n/2^2) + 2 c n \\
&= 2^3 T(n/2^3) + 3 c n \\
&\dots\dots\dots \\
&= 2^k T(n/2^k) + k c n && [\text{Assume that } 2^k = n \Rightarrow k = \log n] \\
&= n T(1) + c n \log n \\
&= a n + c n \log n \\
&= \mathbf{O(n \log n)}
\end{aligned}$$

Best Case, Average Case and Worst Case Complexity of Merge Sort = **O(n log n)**

o Divide and Conquer Matrix Multiplication

- **Native Matrix Multiplication Algorithm**

for i = 0 to n - 1 do

for j = 0 to n - 1 do

for k = 0 to n-1 do

$$C[i, j] = C[i, j] + (A[i, k] * A[k, j])$$

- Native matrix multiplication complexity = $O(n^3)$

- **Divide and Conquer Matrix Multiplication Algorithm**

1. We have to compute the product of 2 nxn matrices A and B.
2. Assume that n is the power of 2. That is $n=2^k$
If n is not a power of 2, then enough rows and columns of 0's can be added to both A and B so that the resulting dimensions are the power of two.
3. Then partition A and B into 4 square matrices, each of size $n/2 \times n/2$
4. AB can be computed using the formula

$$\begin{aligned}
C_{11} &= A_{11} B_{11} + A_{12} B_{21} \\
C_{12} &= A_{11} B_{12} + A_{12} B_{22} \\
C_{21} &= A_{21} B_{11} + A_{22} B_{21} \\
C_{22} &= A_{21} B_{12} + A_{22} B_{22}
\end{aligned}$$
5. If $n=2$, these formulas are computed using a multiplication operation for the elements of A and B
6. If $n>2$, the elements of C can be computed using matrix multiplications and addition operations applied to the matrices of size $n/2 \times n/2$
7. This algorithm will continue applying itself to smaller sized sub-matrices until n becomes suitably small($n=2$) so that the product is computed directly.

- **Complexity**

- For multiplying two matrices of size $n \times n$, we make 8 recursive calls above, each on a matrix with size $n/2 \times n/2$.
- Addition of two matrices takes $O(n^2)$ time.
- Time complexity = $8 T(n/2) + O(n^2)$
= **$O(n^3)$** && [By Master's Theorem]

O Strassen's Matrix Multiplication

• Algorithm

1. A and B are the matrices with dimension $n \times n$
2. If n is not a power of 2, then enough rows and columns of 0's can be added to both A and B so that the resulting dimensions are the power of two.
3. Partition A and B into 4 square matrices of size $n/2 \times n/2$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

A B

a, b, c and d are sub-matrices of A, of size $n/2 \times n/2$

e, f, g and h are sub-matrices of B, of size $n/2 \times n/2$

4. Compute 7 $n/2 \times n/2$ matrices

$$P_1 = a (f - h)$$

$$P_2 = h (a + b)$$

$$P_3 = e (c + d)$$

$$P_4 = d (g - e)$$

$$P_5 = (a + d) (e + h)$$

$$P_6 = (b - d) (g + h)$$

$$P_7 = (a - c) (e + f)$$

It requires 7 matrix multiplications and 10 matrix additions and subtractions

5. Then compute C

$$C = \begin{bmatrix} C_1 & C_2 \\ C_3 & C_4 \end{bmatrix}$$

$$C_1 = P_4 + P_5 + P_6 - P_2$$

$$C_2 = P_1 + P_2$$

$$C_3 = P_3 + P_4$$

$$C_4 = P_1 - P_3 + P_5 - P_7$$

• Complexity

- For multiplying two matrices of size $n \times n$, we make 7 matrix multiplications and 10 matrix additions and subtractions
- Addition/Subtraction of two matrices takes $O(n^2)$ time.
- Time complexity = $7 T(n/2) + O(n^2)$
 $= O(n^{\log_2 7}) = O(n^{2.81})$ [By Master's Theorem]

$$T(n) = \begin{cases} b & \text{if } n < 2 \\ 7 T(n/2) + c n^2 & \text{Otherwise} \end{cases}$$

$$\begin{aligned} T(n) &= 7 T(n/2) + c n^2 \\ &= 7^2 T(n/2^2) + 7 c n^2/4 + c n^2 \\ &= 7^3 T(n/2^3) + 7^2 c n^2/4^2 + 7 c n^2/4 + c n^2 \\ &\dots\dots\dots \\ &= 7^k T(n/2^k) + (7^{k-1}/4^{k-1}) c n^2 + \dots\dots\dots + (7/4) c n^2 + c n^2 \\ &= 7^k T(n/2^k) + [1 + (7/4) + \dots\dots\dots + (7^{k-1}/4^{k-1})] c n^2 \\ &\leq 7^k T(n/2^k) + [1 + (7/4) + \dots\dots\dots] c n^2 \end{aligned}$$

$$\begin{aligned}
&= 7^k T(n/2^k) + [1/(1-(7/4))] c n^2 \\
&= 7^{\log n} T(1) - [4/3] c n^2 \\
&= n^{\log 7} O(1) - [4/3] c n^2 \\
&= O(n^{\log 7}) = O(n^{2.81})
\end{aligned}$$

[Assume that $n/2^k = 1 \Rightarrow k = \log n$]

O University Questions

1. Explain divide and conquer strategy
2. Give a control abstraction for Divide and Conquer method. Explain with an example
3. Write and explain merge sort algorithm using divide and conquer strategy. Also analyse the complexity
4. Write an algorithm for two-way merge sort. Calculate the time complexity of merge sort
5. Illustrate the divide and conquer approach by applying 2 way merge sort for the input array: [15,12,14,17,11,13,12,16]. Write the recurrence for merge sort and give the complexity
6. Write and explain merge sort algorithm using divide and conquer strategy using the data {30, 19, 35, 3, 9, 46, 10}. Also analyse the time complexity.
7. Explain Strassen's matrix multiplication with example and analyse its complexity
8. Multiply the following two matrices using Strassen's Matrix Multiplication Algorithm

$$A = \begin{bmatrix} 6 & 8 \\ 9 & 7 \end{bmatrix} \quad B = \begin{bmatrix} 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Answer:

1. A and B are the matrices with dimension 2x2
2. Here n is a power of 2
3. Partition A and B into 4 square matrices of size $n/2 \times n/2 = 1 \times 1$

$$A = \begin{bmatrix} 6 & 8 \\ 9 & 7 \end{bmatrix} \quad B = \begin{bmatrix} 2 & 5 \\ 3 & 6 \end{bmatrix}$$

$$a=6, b=8, c=9, d=7$$

$$e=2, f=5, g=3, h=6$$

4. Compute 7 $n/2 \times n/2$ matrices

$$P_1 = a (f - h) = 6 (5-6) = -6$$

$$P_2 = h (a + b) = 6(6+8) = 84$$

$$P_3 = e (c + d) = 2(9+7) = 32$$

$$P_4 = d (g - e) = 7(3-2) = 7$$

$$P_5 = (a + d) (e + h) = (6+7)(2+6) = 104$$

$$P_6 = (b - d) (g + h) = (8-7)(3+6) = 9$$

$$P_7 = (a - c) (e + f) = (6-9)(2+5) = -21$$

5. Then compute

$$C_1 = P_4 + P_5 + P_6 - P_2 = 7+104+9-84 = 36$$

$$C_2 = P_1 + P_2 = -6+84 = 78$$

$$C_3 = P_3 + P_4 = 32+7 = 39$$

$$C_4 = P_1 - P_3 + P_5 - P_7 = -6-32+104+21 = 87$$

$$C = \begin{bmatrix} C_1 & C_2 \\ C_3 & C_4 \end{bmatrix} = \begin{bmatrix} 36 & 78 \\ 39 & 87 \end{bmatrix}$$

9. Compare the time complexity of Strassen's matrix multiplication with ordinary matrix multiplication. Which algorithm run faster? Why?
10. Why Strassen's matrix multiplication algorithm is better than traditional divide and conquer algorithm for multiplying two square matrices? What is the recurrence for the number of computational steps taken by Strassen's algorithm and its time complexity?
11. Write an algorithm based on divide-and-conquer strategy to search an element in a given list. Assume that the elements of list are in sorted order.
12. Strassen's multiplication method is used to multiply two nxn matrices when n is a power of 2. How it can be modified when n is not a power of 2?

- **Greedy Strategy**

- o **Control Abstraction**

```

Greedy(a, n) //a[1..n] contains n inputs
{
    solution = Φ;
    for i=1 to n do
    {
        x = Select(a);
        if Feasible(solution, x) then
            solution = Union(solution, x);
    }
    return solution;
}
  
```

- Select() selects an input from the array a[] and remove it. The selected input value is assigned to x.
- Feasible() is a Boolean valued function that determines whether x can be included into the solution subset.
- Union() combines x with the solution and updates the objective function.

- **Fractional Knapsack Problem**

- o We are given with n objects and a knapsack(or bag) of capacity m . The object i has weight W_i and profit P_i . If a fraction X_i is placed into the knapsack, then a profit $P_i X_i$ is obtained. The objective is to obtain an optimal solution of the knapsack that maximizes the total profit earned.
- o The total weight of all the chosen objects should not be more than m .
- o Fractional knapsack problem can be stated as

$$\text{Maximize } \sum_{i=1}^n P_i X_i \quad \text{—————} \quad \textcircled{1}$$

$$\text{Subject to } \sum_{i=1}^n W_i X_i \leq m \quad \text{—————} \quad \textcircled{2}$$

$$0 \leq X_i \leq 1 \text{ and } 1 \leq i \leq n \quad \text{—————} \quad \textcircled{3}$$

The profits and weights are positive numbers.

- o A feasible solution is one that satisfies equation 2 and 3.
- o An optimal solution is a feasible solution that satisfies equation 1.
- o In greedy strategy we are arranging the objects in the descending order of profit/weight.

0 Algorithm

```

Algorithm GreedyKnapsack(m, n)
//p[1:n] is the profits and w[1:n] is the weights of n objects such that  $p[i]/w[i] \geq p[i+1]/w[i+1]$ .
{
    for i= 1 to n do
        x[i] = 0.0;           // x[1:n] is the solution vector
        U = m;                // m is the knapsack capacity
        for i=1 to n do
            {
                if w[i] > U then
                    break;
                x[i] = 1.0
                U = U - w[i];
            }
        If  $i \leq n$  then
            x[i] = U / w[i];
    }

```

0 Time Complexity

- The for loop will execute maximum n times. So the time complexity = **O(n)**

0 University Questions

1. Explain Greedy Approach. Write the general greedy algorithm
2. Give the control abstraction of Greedy strategy
3. Write the control abstraction for Greedy design technique. Give a greedy algorithm for fractional knapsack problem.
4. State fractional knapsack problem. Give an algorithm for fractional knapsack problem using greedy strategy
5. Find an optimal solution to the fractional knapsack problem for an instance with number of items 7, Capacity of the sack $W=15$, profit associated with the items $(p_1, p_2, \dots, p_7) = (10, 5, 15, 7, 6, 18, 3)$ and weight associated with each item $(w_1, w_2, \dots, w_7) = (2, 3, 5, 7, 1, 4, 1)$.

Answer:

- Arrange the objects in the descending order of profit/weight

$i = \{1, 2, 3, 4, 5, 6, 7\}$
 $P = \{10, 5, 15, 7, 6, 18, 3\}$
 $W = \{2, 3, 5, 7, 1, 4, 1\}$

$P_i/W_i = \{5, 1.66, 3, 1, 6, 4.5, 3\}$

Now the i, P and W arrays are

$i = \{5, 1, 6, 3, 7, 2, 4\}$
 $P = \{6, 10, 18, 15, 3, 5, 7\}$
 $W = \{1, 2, 4, 5, 1, 3, 7\}$

Initially $U=m=15$

Item	Pi	Wi	Xi	U = U - Wi
5	6	1	1	14
1	10	2	1	12
6	18	4	1	8
3	15	5	1	3
7	3	1	1	2
2	5	3	2/3	0
4	7	7	0	0

- Total weight of the chosen objects are
 $\sum_{i=1}^n W_i X_i = 2 \times 1 + 3 \times 2/3 + 5 \times 1 + 7 \times 0 + 1 \times 1 + 4 \times 1 + 1 \times 1 = 15$
 - Profit earned is $\sum_{i=1}^n P_i X_i = 10 \times 1 + 5 \times 2/3 + 15 \times 1 + 7 \times 0 + 6 \times 1 + 18 \times 1 + 3 \times 1 = 55.33$
 - Solution vector $\mathbf{X} = \{1, 2/3, 1, 0, 1, 1, 1\}$
6. Find the optimal solution for the following fractional Knapsack problem. Given number of items(n)=4, capacity of sack(m) = 60, $W = \{40, 10, 20, 24\}$ and $P = \{280, 100, 120, 120\}$

Answer:

m = 60

n = 4

i $\rightarrow \{1, 2, 3, 4\}$

P = {280, 100, 120, 120}

W = {40, 10, 20, 24}

P/W = {7, 10, 6, 5}

Sort $p[i]/w[i] \geq p[i+1]/w[i+1]$

i $\rightarrow \{2, 1, 3, 4\}$

P = {100, 280, 120, 120}

W = {10, 40, 20, 24}

Item	Pi	Wi	Xi	U = U-Wi
2	100	10	1	50
1	280	40	1	10
3	120	20	1/2	0
4	120	24	0	

Total Profit = $\sum P_i * X_i$
 $= 100 \times 1 + 280 \times 1 + 120 \times 1/2 = 440$

Solution vector $\mathbf{X} = \{1, 1, 1/2, 0\}$

7. Find optimal solution for the following Knapsack problem
 n=3, m=20, $W = \{18, 15, 10\}$, $P = \{25, 24, 20\}$
8. Consider the following instance of Fractional Knapsack problem with 3 objects. The capacity of the knapsack is 20 units. The weights and profits of the 3 items respectively are represented by the vectors $(w_1, w_2, w_3) = (18, 15, 10)$ and $(p_1, p_2, p_3) = (25, 24, 15)$. Using a greedy strategy compute the optimal solution to this instance.
9. A knapsack that can carry a maximum weight of 60. There are 4 items with weights {20, 30, 40, 70} and values {70, 80, 90, 200}. What is the maximum value of the items you can carry using the knapsack?
10. Apply greedy algorithm for fractional knapsack to find the optimal ordering for loading the items in the knapsack. Let the knapsack capacity, M=15

Items	Weight	Profit
1	8	24
2	9	18
3	5	20

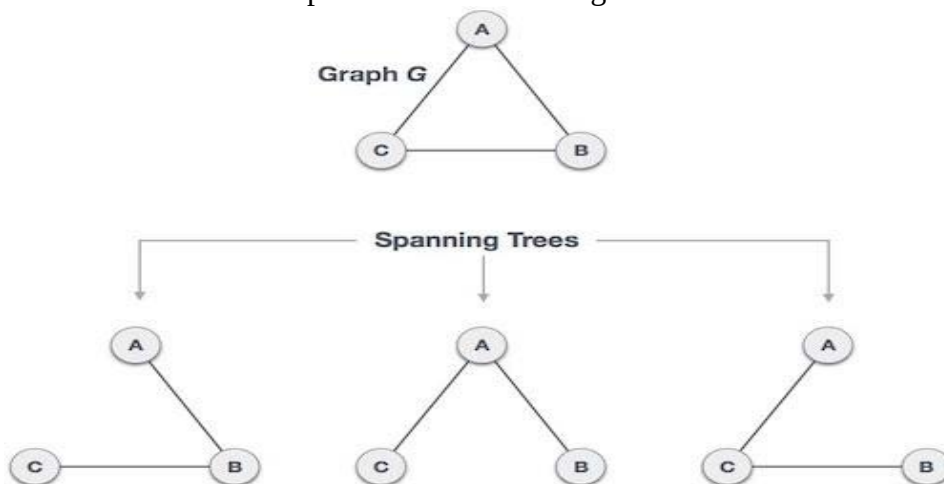
11. Find the optimal solution for the following Fractional Knapsack problem. Given the number of items(n) = 7, capacity of sack(m) = 15, $W = \{1, 3, 5, 4, 1, 3, 2\}$ and $P = \{10, 15, 7, 8, 9, 4\}$

12. Find an optimal solution to the fractional knapsack instance $n=5$, $m=60$, $(p_1, p_2, \dots, p_5) = (30, 20, 100, 90, 160)$ and $(w_1, w_2, \dots, w_5) = (5, 10, 20, 30, 40)$, Where m is the knapsack capacity, p_i is the profit of i^{th} item and w_i is the weight of i^{th} item.
13. For the given set of items and knapsack capacity = 60 kg, find the optimal solution for the fractional knapsack problem making use of greedy approach.

Item	Weight	Value
1	5	30
2	10	40
3	15	45
4	22	77
5	25	90

• Spanning Trees

- o A spanning tree is a subset of undirected connected Graph $G=(V,E)$, which has all the vertices covered with minimum possible number of edges.



o Properties of Spanning Tree

- A connected graph G can have more than one spanning tree.
- All possible spanning trees of graph G , have the same number of edges and vertices.
- The spanning tree does not have any cycle (loops)
- Removing one edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is **minimally connected**
- Adding one edge to the spanning tree will create a circuit or loop, i.e. the spanning tree is **maximally acyclic**
- Spanning tree has $n-1$ edges, where n is the number of nodes.
- o Maximum number of Spanning Trees of a graph with n nodes
 - Complete Graph: n^{n-2}
 - Other Graphs
 1. Create Adjacency Matrix for the given graph.
 2. Replace all the diagonal elements with the degree of nodes.
 3. Replace all non-diagonal 1's with -1.
 4. Total number of spanning tree for that graph = Co-factor for any element in that matrix.

o **Minimum Spanning Tree (MST)**

- In a weighted graph, a minimum spanning tree is a spanning tree that has minimum weight than all other spanning trees of the same graph.
- In real-world situations, this weight can be measured as distance, congestion, traffic load or any arbitrary value denoted to the edges.
- Minimum Spanning-Tree Algorithms
 - Prim's Algorithm
 - Kruskal's Algorithm

o **Application of Spanning Tree**

- Civil Network Planning
- Computer Network Routing Protocol
- Cluster Analysis
- Handwriting Recognition
- Image Segmentation

• **Minimal Cost Spanning Tree Computation**

o **Kruskal's Algorithm.**

- Kruskal's Algorithm builds the spanning tree by adding edges one by one into a growing spanning tree.
- Kruskal's algorithm follows greedy approach as in each iteration it finds an edge which has least weight and add it to the growing spanning tree
- In this algorithm, the edges of the graph are considered in the increasing order of cost.
- If the selected edge will form a cycle, then discard it.
- This selection process continues until there are $n-1$ edges.

Algorithm Kruskals(E, cost, n, t)

```
{
    Construct a heap out of edge costs using Heapify();
    for i=1 to n do
        parent[i] = -1;
    i=0;
    mincost = 0.0;
    while (i < n-1) and (heap not empty) do
    {
        Delete a minimum cost edge (u, v) from the heap and reheapify using Adjust();
        j = Find(u);
        k = Find(v);
        if j ≠ k then
        {
            i = i+1;
            t[i, 1] = u;    t[i, 2] = v;
            mincost = mincost + cost[u, v];
            Union(j, k);
        }
    }
    if i ≠ n-1 then
        Write ("No Spanning Tree");
    else
        return mincost;
}
```

E is the set of edges and n is the number of vertices in G .

$\text{cost}[u,v]$ is the cost of edge (u, v) .

t is the set of edges in the minimum cost spanning tree.

The final cost is returned

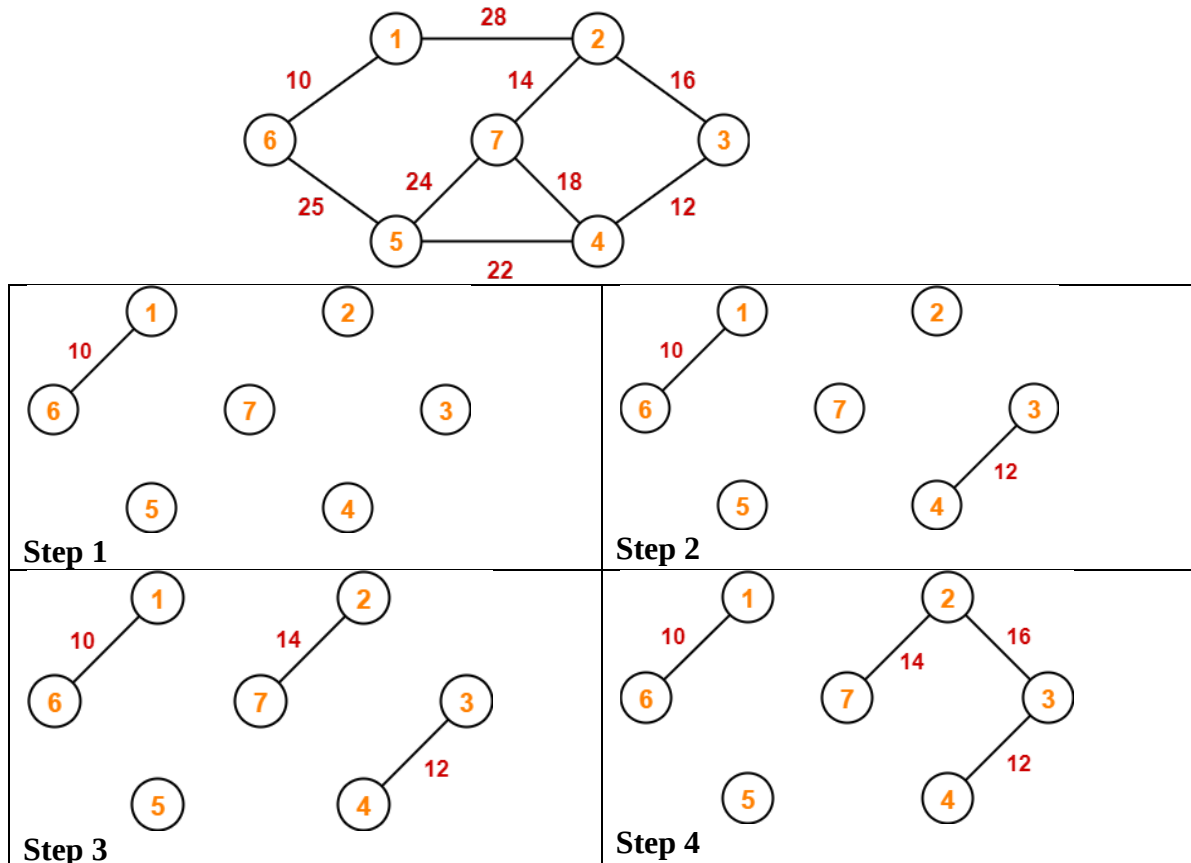
- $\text{Heapify}()$ is used to construct a minheap based on the edge cost of G .
- $\text{Adjust}()$ is used to reconstruct a minheap if there is a deletion occurs.
- Initially all vertices belong to different sets. $\text{Find}()$ returns the set number of that particular vertex. j and k are the set number of vertex u and v respectively.
- If $j=k$ means vertex u and v belong to the same set. Inclusion of (u, v) should definitely form a cycle. So discard it.
- If $j \neq k$ means vertex u and v belong to different set. Inclusion of (u, v) will not form a cycle. So add it to the minimum spanning tree edge list.
- Finally there are $n-1$ edges, then return it. Otherwise there is no spanning tree.

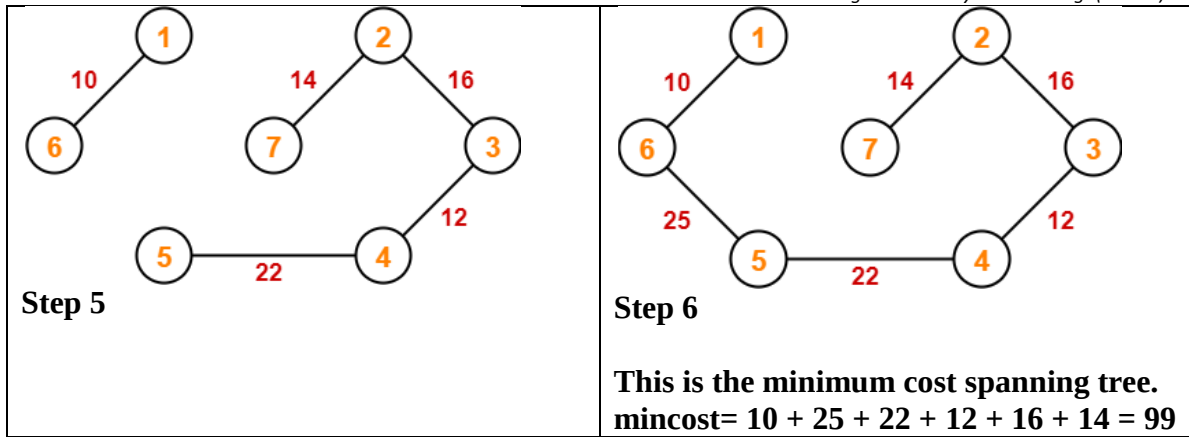
o Complexity

- The edges are maintained as a minheap, then the next edge to consider can be obtained in $O(\log |E|)$ time.
- Construction of heap itself takes $O(|E|)$ time.
- Overall complexity of Kruskal's algorithm is $O(|E| \log |E|)$.

o Example

- Construct the minimum spanning tree for the given graph using Kruskal's Algorithm





o University Questions

1. Define spanning tree of a graph.
2. Write the total number of spanning trees possible for a complete graph with 4 vertices

Answer:

Total number of spanning trees possible for a complete graph with n vertices = n^{n-2}

Total number of spanning trees possible for a complete graph with 6 vertices = $4^{4-2} = 4^2 = 16$

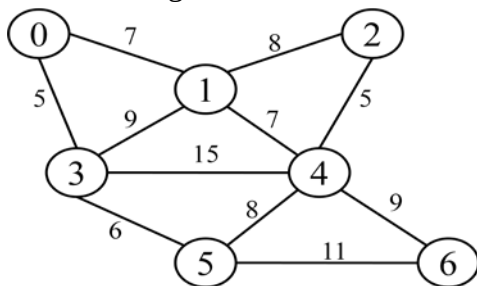
3. Write the total number of spanning trees possible for a complete graph with 6 vertices.

Answer:

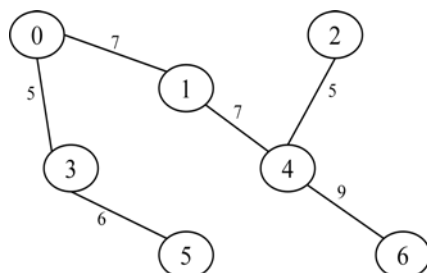
Total number of spanning trees possible for a complete graph with n vertices = n^{n-2}

Total number of spanning trees possible for a complete graph with 6 vertices = $6^{6-2} = 6^4 = 1296$

4. Define spanning tree of a graph. Write the total number of spanning trees possible for a complete graph with 4 vertices.
5. Write the Kruskal's algorithm for Minimum Spanning Tree. Analyse its complexity
6. Write the Kruskal's algorithm for finding minimum cost spanning tree and explain the Kruskal's algorithm with an example. Analyse the complexity of the algorithm.
7. Compute the Minimum Spanning Tree and its cost for the following graph using Kruskal's Algorithm. Indicate each step clearly

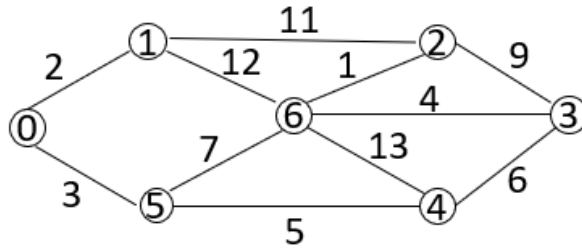


Answer:

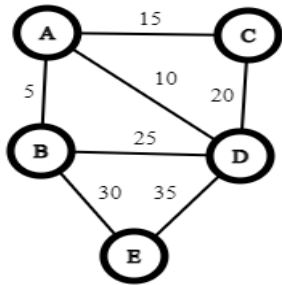


Cost = 39

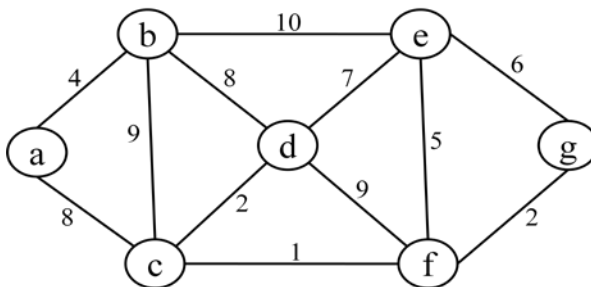
8. Apply Kruskal's algorithm for finding minimum cost spanning tree.



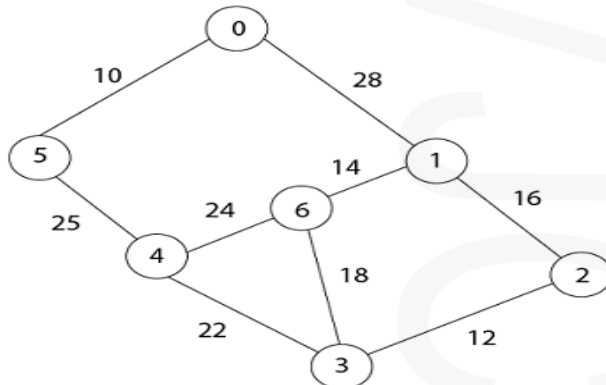
9. Define spanning tree of a graph. Write the total number of spanning trees possible for a complete graph with 4 vertices.



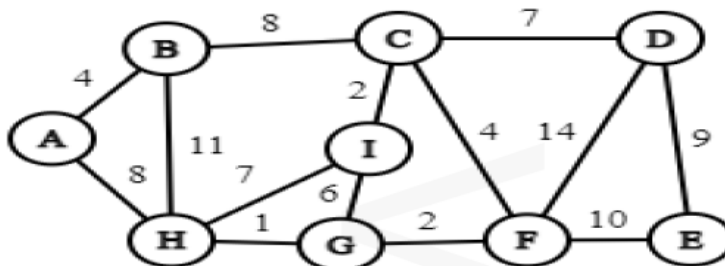
10. Apply Kruskal's algorithm on the graph given below.



11. Find the minimum cost spanning tree for the given graph using Kruskal's algorithm



12. Apply Kruskal algorithm to find minimum cost spanning tree for the following graph

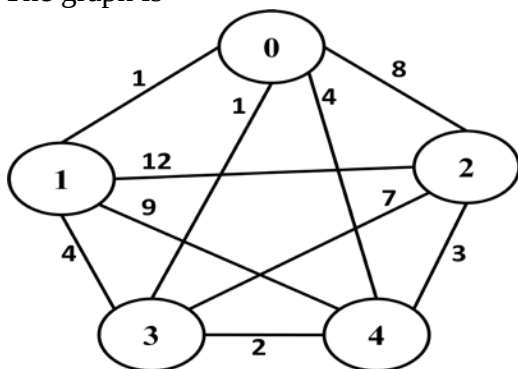


13. Let (u,v) be a minimum-weight edge in a graph G . Show that (u,v) belongs to some minimum spanning tree of G .
- Suppose that T is a Minimum Spanning Tree, which does not include the smallest edge, E .
 - Add E to T . Now a circle C is formed.
 - This graph will remain connected if an edge is removed from the circle C .
 - So remove an edge E' (except E) from C which also belongs to T
 - This operation would result a new spanning tree whose weight is $\leq$ weight of T .
 - We have a contradiction. Hence, proved.
14. Let G be a weighted undirected graph with distinct positive edge weights. If every edge weight is increased by same value, will the minimum cost spanning tree change. Justify your answer
- **The Minimum Spanning Tree doesn't change.** In Kruskal's algorithm, we will sort the edges first. If we increase all weights, then order of edges won't change. So, MST does not change.
15. Consider a complete undirected graph with vertex set $\{0, 1, 2, 3, 4\}$. Entry w_{ij} in the matrix W below is the weight of the edge $\{i, j\}$. What is the minimum possible weight of a Spanning Tree T in this graph such that vertex 0 is a leaf node in the tree T ?

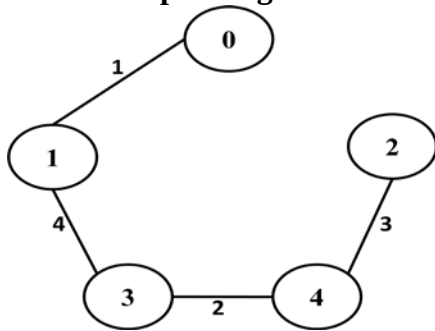
$$W = \begin{pmatrix} 0 & 1 & 8 & 1 & 4 \\ 1 & 0 & 12 & 4 & 9 \\ 8 & 12 & 0 & 7 & 3 \\ 1 & 4 & 7 & 0 & 2 \\ 4 & 9 & 3 & 2 & 0 \end{pmatrix}$$

Answer:

The graph is

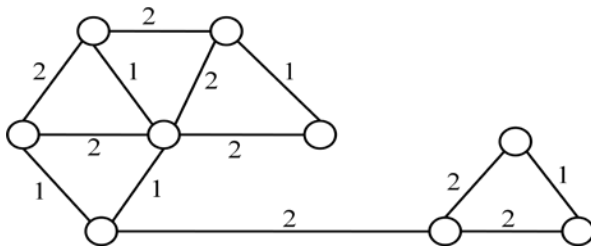


Minimum spanning tree in which vertex 0 is a leaf node is

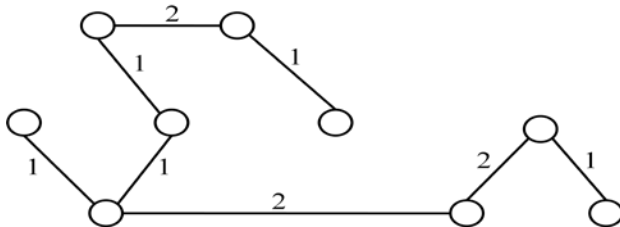


Cost = 10

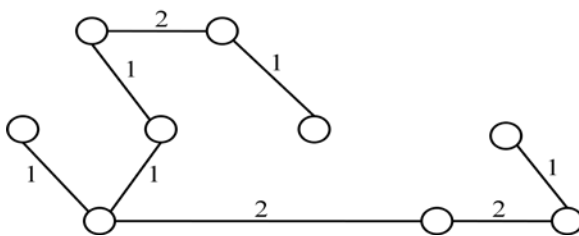
16. Find the number of distinct minimum spanning trees for the weighted graph below



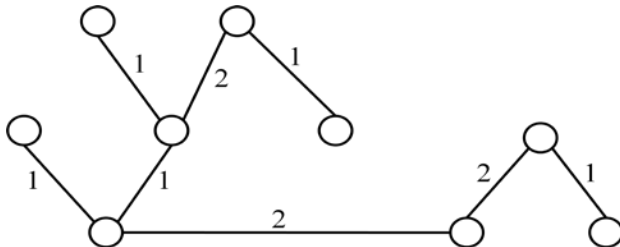
Answer:



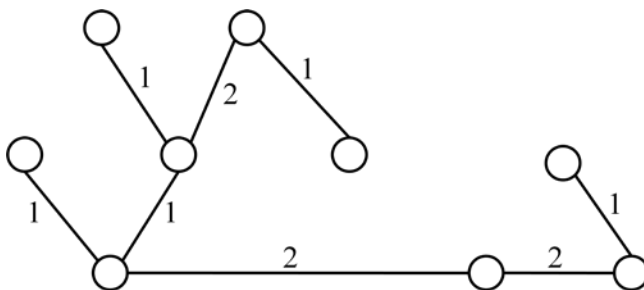
Spanning Tree -1
Cost = 11



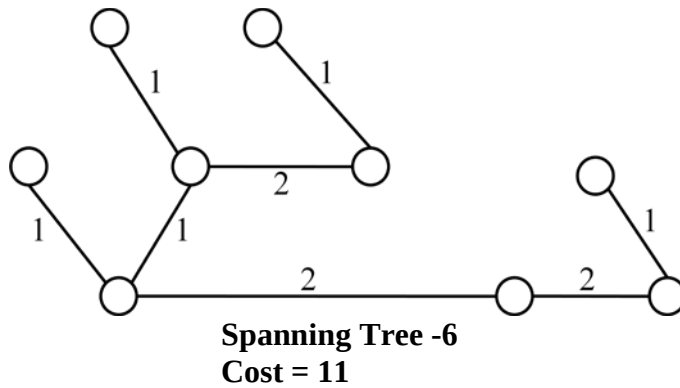
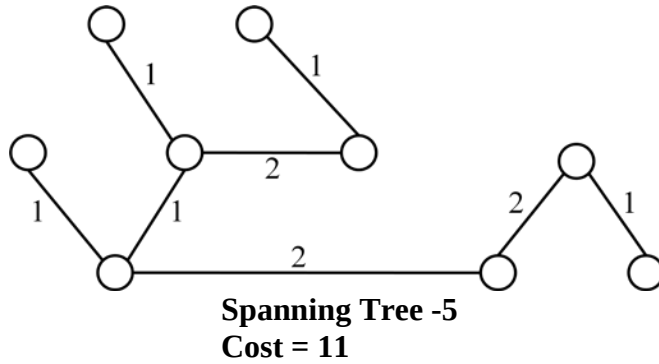
Spanning Tree -2
Cost = 11



Spanning Tree -3
Cost = 11

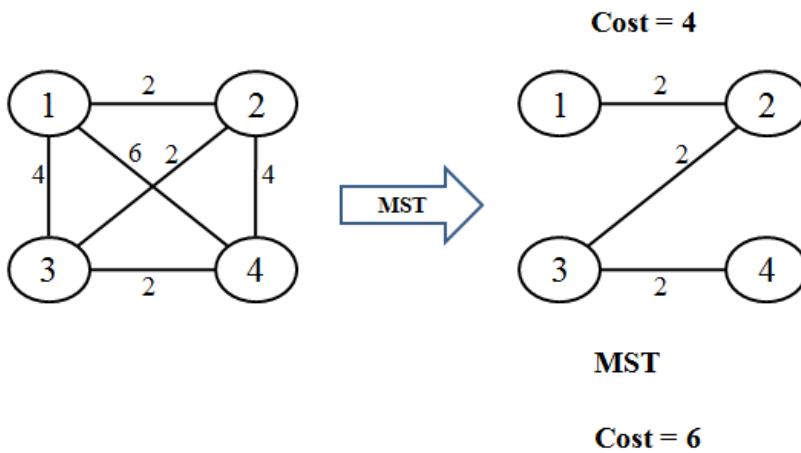
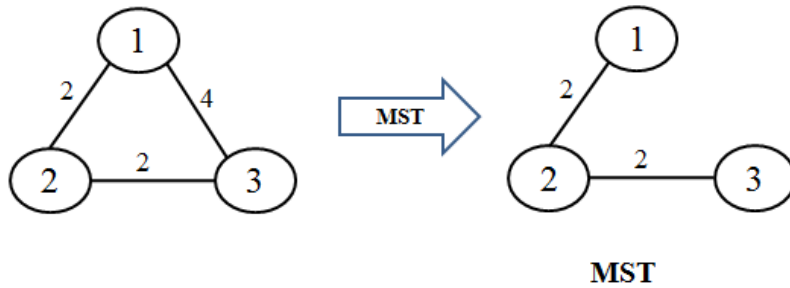


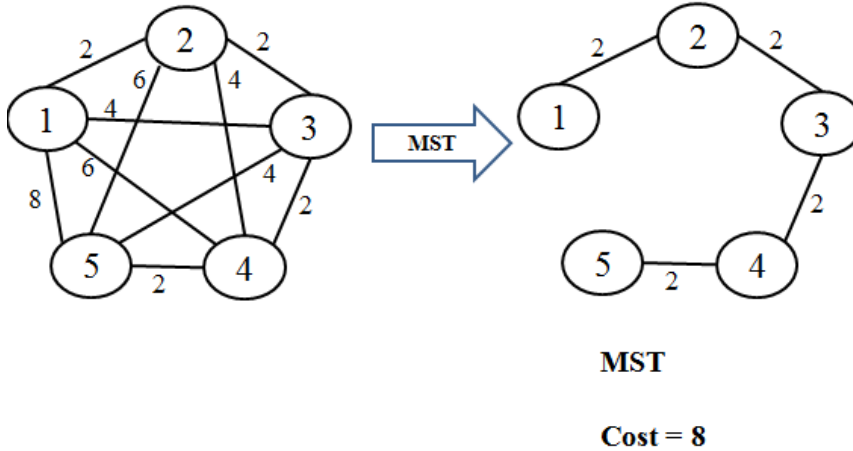
Spanning Tree -4
Cost = 11



17. Consider a weighted complete graph G on the vertex set $\{v_1, v_2, \dots, v_n\}$ such that the weight of the edge (v_i, v_j) is $2|i-j|$. Find the weight of a minimum spanning tree of G .

Answer:





Number of nodes=3

Cost of MST=4

Number of nodes=4

Cost of MST=6

Number of nodes=5

Cost of MST=8

.

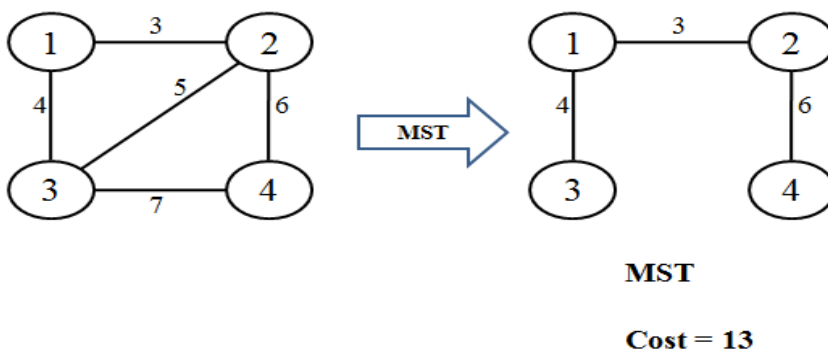
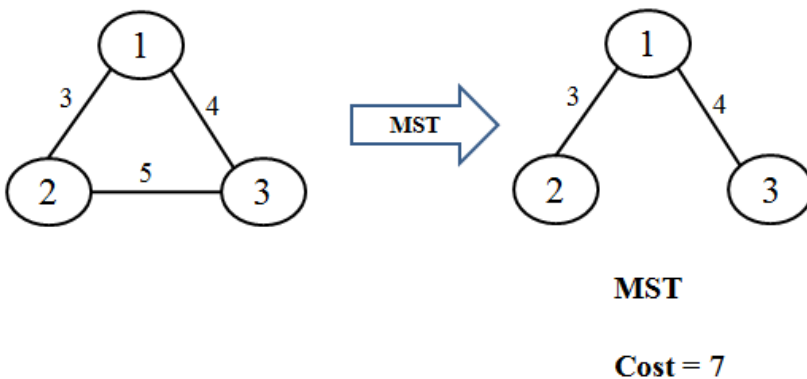
.

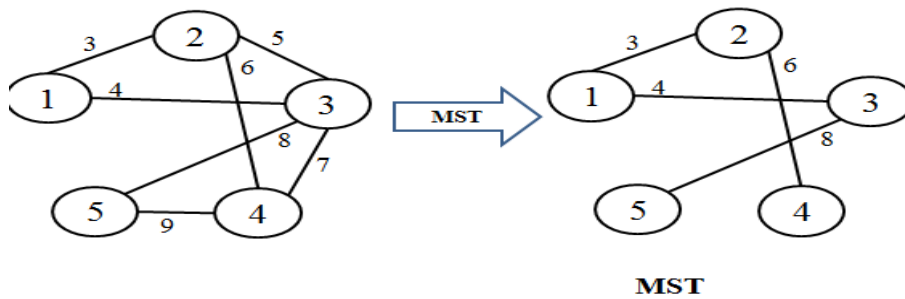
Number of nodes=n

Cost of MST=2n - 2

18. An undirected graph $G=(V, E)$ contains n ($n > 2$) nodes named $v_1, v_2, \dots, v_n$. Two vertices v_i, v_j are connected if and only if $0 < |i - j| \leq 2$. Each edge (v_i, v_j) is assigned a weight $i + j$. What will be the cost of the minimum spanning tree (as a function of n) of such a graph with n nodes?

Answer:





Cost = 21

Number of nodes=3

Cost of MST=3+4=7

Number of nodes=4

Cost of MST=3+4+6=13

Number of nodes=5

Cost of MST=3+4+6+8=21

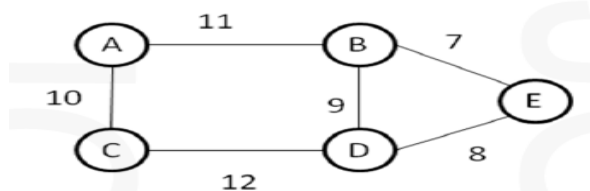
•
•

Number of nodes=n

Cost of MST=n(n-1) + 1

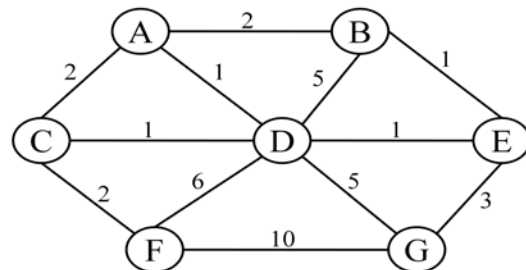
19. Apply these algorithms on the following graph. Let A be the source vertex. Analyse complexity of each algorithm

i) Prim's ii) Kruskal



20. Write down Prim's algorithm and analyse the complexity

21. Apply Prim's algorithm on the following graph. Let A be the source vertex



22. Consider a complete undirected graph with vertex set $\{0, 1, 2, 3, 4\}$. Entry w_{ij} in the matrix W below is the weight of the edge $\{i, j\}$. What is the Cost of the Minimum Spanning Tree T using Prim's Algorithm in this graph such that vertex 0 is a leaf node in the tree T ?

$$W = \begin{pmatrix} 0 & 1 & 8 & 1 & 4 \\ 1 & 0 & 12 & 4 & 9 \\ 8 & 12 & 0 & 7 & 3 \\ 1 & 4 & 7 & 0 & 2 \\ 4 & 9 & 3 & 2 & 0 \end{pmatrix}$$

- **Single Source Shortest Path Algorithms**

- o The shortest path problem is the problem of finding a path between two vertices in a graph such that the sum of the weights of its constituent edges is minimized.

- o Different shortest path problems are:

1. **Single Source Shortest Path Problem:**

- Given a connected weighted graph $G=(V,E)$, find the shortest path from a given source vertex s to every other vertices $(V-\{s\})$ in the graph.
- The weight of any path($w(p)$) is the sum of the weights of its constituted edges.
- The weight of the shortest path from u to $v = \min\{w(p): p \text{ is a path from } u \text{ to } v\}$

2. **Single Destination Shortest Path Problem:** To find shortest paths from all vertices in the directed graph to a single destination vertex v

3. **All Pairs Shortest Path Problem:** To find shortest paths between every pair of vertices in the graph

- o Single Source Shortest Path Algorithms are:

1. Dijkstra's Algorithm
2. Bellman Ford Algorithm

- o **Dijkstra's Algorithm**

- Given a graph and a source vertex S in graph, find shortest paths from S to all vertices in the given graph.

- **Algorithm Dijkstra(G, W, S)**

1. For each vertex v in G
 - 1.1 $\text{distance}[v] = \text{infinity}$
 - 1.2 $\text{previous}[v] = \text{Null}$
2. $\text{distance}[S] = 0$
3. $Q = \text{set of vertices of graph } G$
4. While Q is not empty
 - 4.1 $u = \text{vertex in } Q \text{ with minimum distance}$
 - 4.2 remove u from Q
 - 4.3 for each neighbor v of u which is still in Q
 - 4.3.1 $\text{alt} = \text{distance}[u] + W(u,v)$
 - 4.3.2 if $\text{alt} < \text{distance}[v]$
 - 4.3.2.1 $\text{distance}[v] = \text{alt}$
 - 4.3.2.2 $\text{previous}[v] = u$
5. Return $\text{distance}[], \text{previous}[]$

- **Complexity**

- The complexity mainly depends on the implementation of Q
- The simplest version of Dijkstra's algorithm stores the vertex set Q as an ordinary linked list or array, and extract-minimum is simply a linear search through all vertices in Q . In this case, the running time is $O(E + V^2) = O(V^2)$
- Graph represented using adjacency list can be reduced to $O(E \log V)$ with the help of binary heap.

- **University Questions**

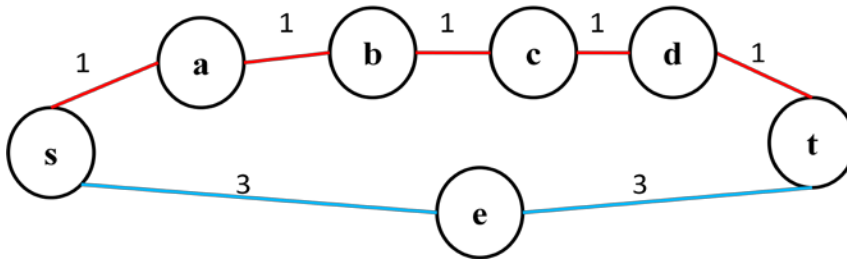
1. Let G be a weighted undirected graph with distinct positive edge weights. If every edge weight is increased by same value, will the shortest path between any pair of vertices change. Justify your answer

Answer:

- **The shortest path may change.**
 - There may be different paths from s to t .
 - Let shortest path(Path-1) be of cost 15 and has 5 edges.
 - Let there be another path(Path-2) of cost 25 and has 2 edges.
 - All edge costs are increased by 10.
 - Path-1 cost is increased by 5×10 and becomes $15 + 50 = 65$.
 - Path-2 cost is increased by 2×10 and becomes $25 + 20 = 45$
 - Now Path-1 cost $>$ Path-2 cost
 - So the shortest path may change.
2. In a weighted graph, assume that the shortest path from a source ' s ' to a destination ' t ' is correctly calculated using a shortest path algorithm. Is the following statement true? If we increase weight of every edge by 1, the shortest path always remains same. Justify your answer with proper example.

Answer:

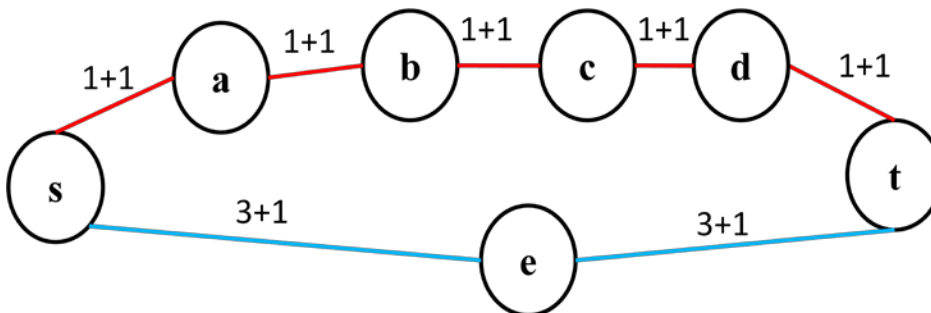
Consider the following graph



Here the Shortest Path is $s-a-b-c-d-t$

Shortest Path Cost = 5

If we increase the weight of every edge by 1, then the graph will become



$s-a-b-c-d-t$ path cost = 10

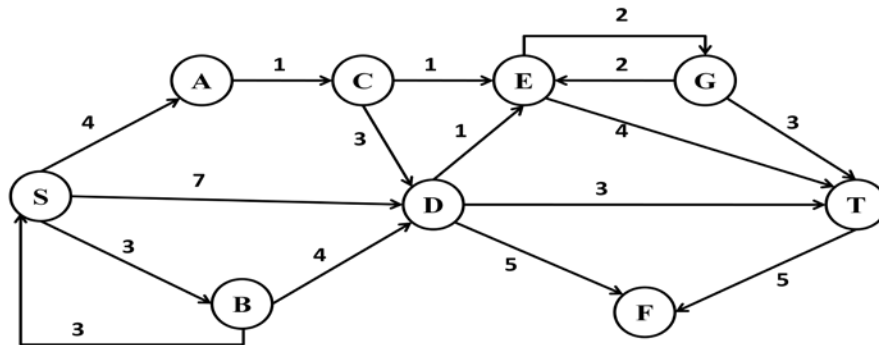
$s-e-t$ path cost = 8

Shortest Path is $s-e-t$

Shortest Path Cost = 8

If we increase weight of every edge by 1, the shortest path may change.

3. Write Dijkstra's Single Source Shortest path algorithm and illustrate with an example. Also analyse its time complexity
4. Find the shortest path from s to all other vertices in the following graph using Dijkstra's Algorithm

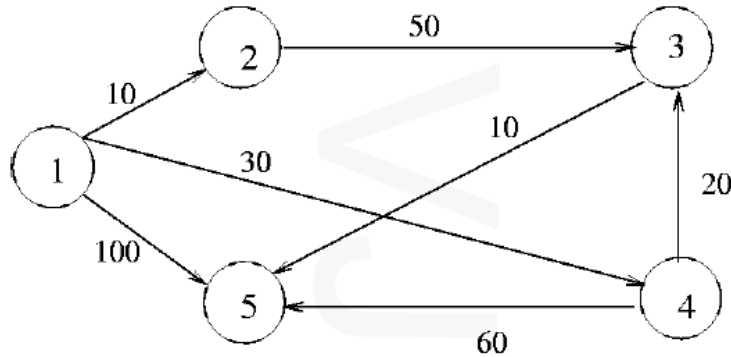


Answer:

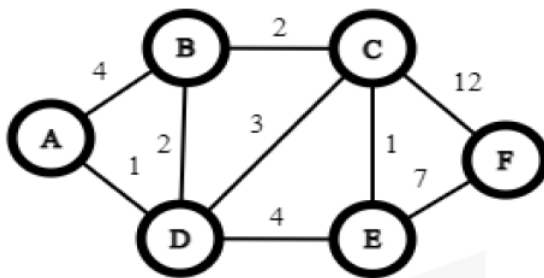
	S	A	B	C	D	E	F	G	T
S	0	α	α	α	α	α	α	α	α
B		4	3	α	7	α	α	α	α
A		4		α	7	α	α	α	α
C				5	7	α	α	α	α
E					7	6	α	α	α
D					7		α	8	10
G							12	8	10
T							12		10
F							12		
		S	S	A	S	C	D	E	E

	Shortest Path	Shortest Distance
S-A	S-A	4
S-B	S-B	3
S-C	S-A-C	5
S-D	S-D	7
S-E	S-A-C-E	6
S-F	S-D-F	12
S-G	S-A-C-E-G	8
S-T	S-A-C-E-T	10

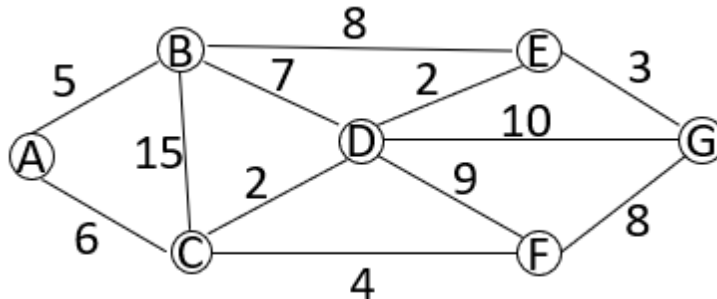
5. Find the minimum path from the source vertex 1 to all other nodes using Dijkstra's algorithm.



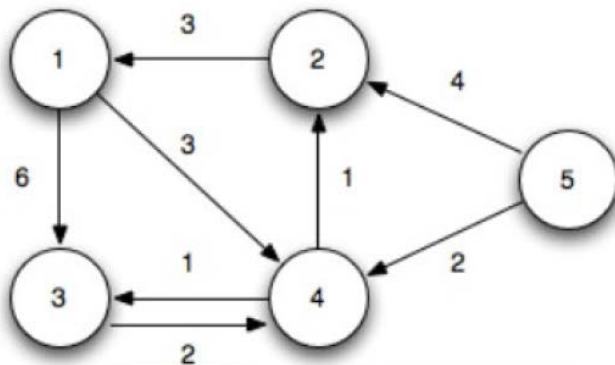
6. Apply Dijkstra's algorithm for single source shortest path to solve the following graph. Assume the source as node A.



7. Apply Dijkstra's algorithm for finding the shortest path from vertex A to all other vertices



8. Write the pseudo code for Dijkstra's algorithm. Compute the shortest distance from vertex 1 to all other vertices using Dijkstra's algorithm



9. Is it possible to find all pairs of shortest paths using Dijkstra's algorithm? Justify